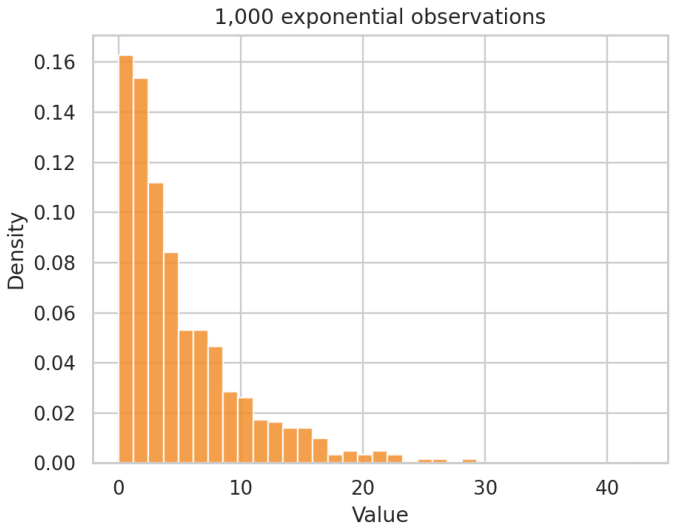
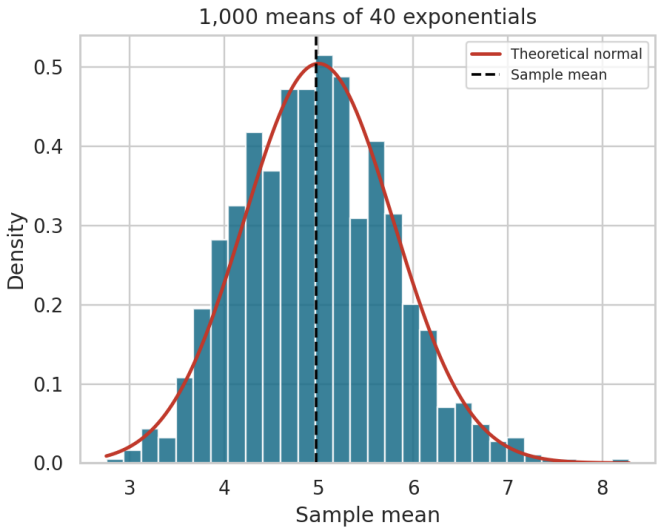
Statistical Inference Course Project

Part 1 - Simulation Exercise

A reproducible simulation generated 1,000 samples, each containing 40 independent exponential observations with rate lambda=0.2. The theoretical mean of an exponential observation is 5; the theoretical mean of a sample mean is also 5. The theoretical variance of a mean of 40 observations is (1/lambda^2)/40 = 0.625.

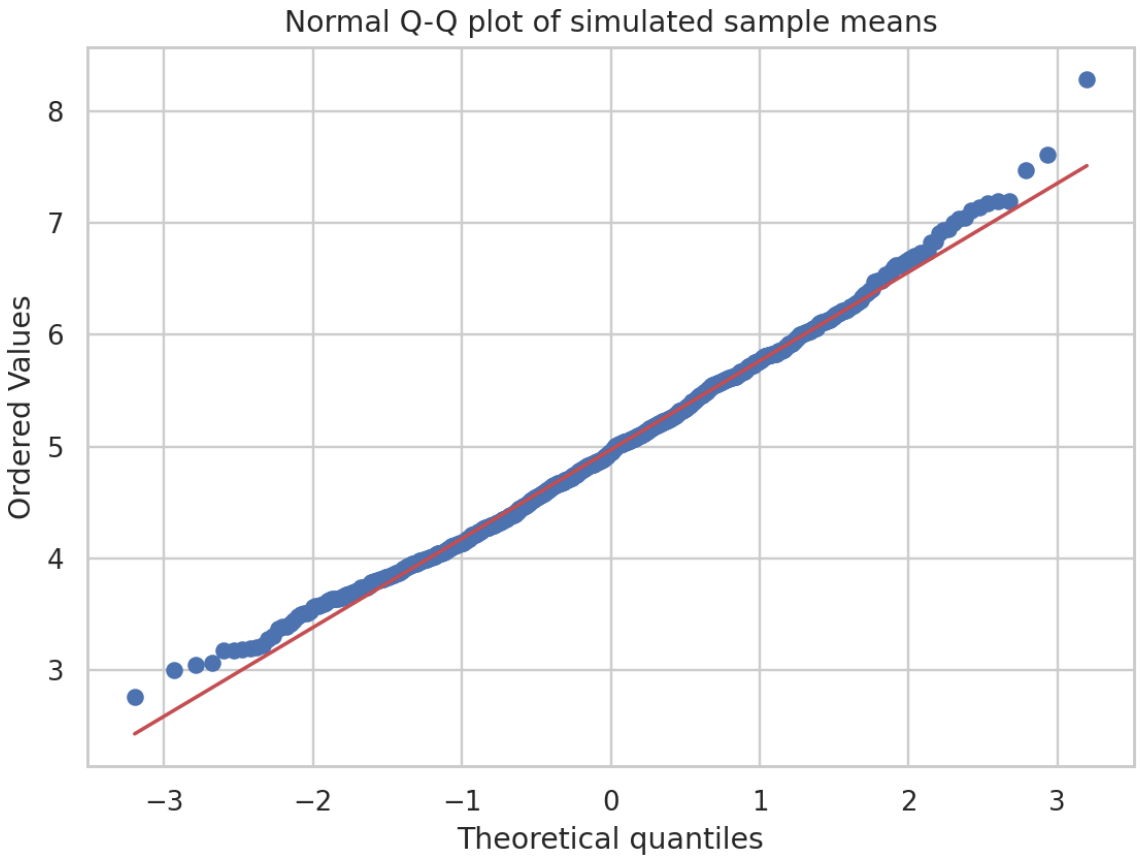




Quantity	Simulated	Theoretical
Mean of sample means	4.971	5.000
Variance of sample means	0.633	0.625
SD of sample means	0.796	0.791

The simulated center and spread are close to their theoretical values. The left panel retains the strong right skew of an exponential distribution; the distribution of averages is much more symmetric and agrees closely with the theoretical normal curve, as predicted by the Central Limit Theorem.

Part 1 - Normal approximation

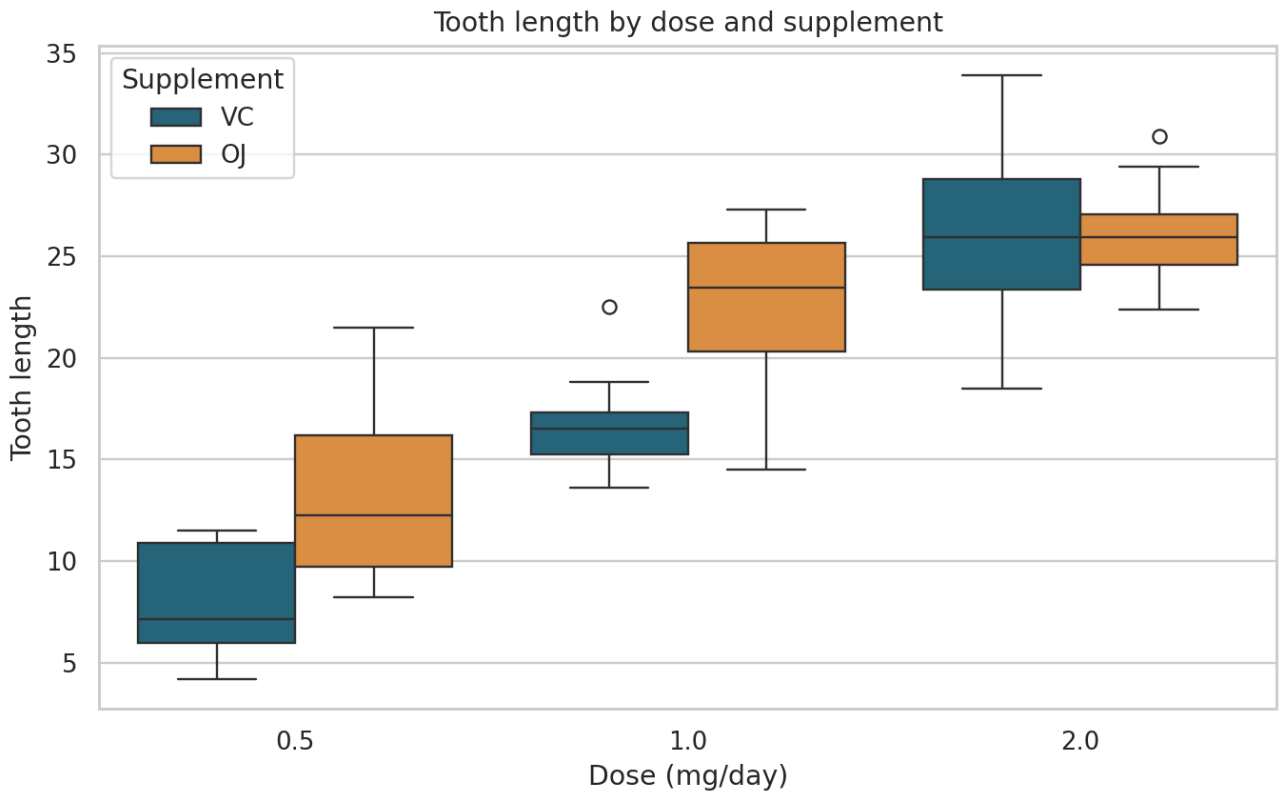


The Q-Q points lie close to the reference line, with modest tail variation expected from 1,000 simulations. Together with the histogram overlay and the agreement of mean and variance, this supports an approximately normal sampling distribution for the mean of 40 exponentials. Independence and identical exponential sampling are required; the result is a simulation demonstration rather than a proof.

Part 2 - Basic Inferential Data Analysis

The ToothGrowth data contain 60 guinea pig observations: tooth length, supplement type (orange juice or vitamin C), and dose (0.5, 1.0, or 2.0 mg/day), with 10 observations per supplement-dose group.

Part 2 - Results and conclusions



Contrast	Difference	95% CI	p (Bonf.)
OJ@0.5 - VC@0.5	5.25	[1.72, 8.78]	0.0382
OJ@1.0 - VC@1.0	5.93	[2.80, 9.06]	0.00623
OJ@2.0 - VC@2.0	-0.08	[-3.80, 3.64]	1
dose 1.0 - dose 0.5	9.13	[6.28, 11.98]	7.61e-07
dose 2.0 - dose 1.0	6.36	[3.73, 9.00]	0.000114
dose 2.0 - dose 0.5	15.50	[12.83, 18.16]	2.64e-13

Welch two-sample t intervals allow unequal variances. After Bonferroni adjustment for six planned comparisons, OJ has higher mean length than VC at doses 0.5 and 1.0, while no difference is supported at dose 2.0. Mean length rises at each higher dose when supplement groups are combined. Conclusions assume independent animals, approximately normal group distributions, and measurements sampled under comparable experimental conditions. Small group sizes limit diagnostics, and the comparisons do not by themselves establish how results generalize beyond the experiment.

Supporting appendix - Complete Python analysis

```
from pathlib import Path
import json
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path(__file__).resolve().parent
FIG = ROOT / "figure"
FIG.mkdir(exist_ok=True)
rng = np.random.default_rng(20261007)

# Part 1: 1,000 averages of 40 Exp(rate=0.2) observations.
rate, n, sims = 0.2, 40, 1000
raw = rng.exponential(scale=1/rate, size=sims)
means = rng.exponential(scale=1/rate, size=(sims, n)).mean(axis=1)
theory_mean = 1/rate
theory_var = 1/(rate**2 * n)

sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
axes[0].hist(raw, bins=35, color="#F28E2B", alpha=.85, density=True)
axes[0].set(title="1,000 exponential observations", xlabel="Value", ylabel="Density")
axes[1].hist(means, bins=30, color="#176B87", alpha=.85, density=True)
x = np.linspace(means.min(), means.max(), 300)
axes[1].plot(x, stats.norm.pdf(x, theory_mean, np.sqrt(theory_var)), color="#C0392B", lw=2, label="Theoretical normal")
axes[1].axvline(means.mean(), color="black", ls="--", label="Sample mean")
axes[1].set(title="1,000 means of 40 exponentials", xlabel="Sample mean", ylabel="Density")
axes[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(FIG/"simulation_distributions.png", dpi=170); plt.close(fig)

fig, ax = plt.subplots(figsize=(6.8, 5.2))
stats.probplot(means, dist="norm", plot=ax)
ax.set_title("Normal Q-Q plot of simulated sample means")
fig.tight_layout(); fig.savefig(FIG/"simulation_qq.png", dpi=170); plt.close(fig)

# Part 2: ToothGrowth.
tg = pd.read_csv(ROOT/"data"/"ToothGrowth.csv").drop(columns=["rownames"])
tg["dose"] = tg["dose"].astype(str)

fig, ax = plt.subplots(figsize=(8.2, 5.2))
sns.boxplot(data=tg, x="dose", y="len", hue="supp", palette=["#176B87", "#F28E2B"], ax=ax)
ax.set(title="Tooth length by dose and supplement", xlabel="Dose (mg/day)", ylabel="Tooth length")
ax.legend(title="Supplement")
fig.tight_layout(); fig.savefig(FIG/"toothgrowth_boxplot.png", dpi=170); plt.close(fig)

def welch(a, b, label_a, label_b):
```

Supporting appendix - Code continued

```
    a, b = np.asarray(a, float), np.asarray(b, float)
    diff = a.mean()-b.mean()
    se = np.sqrt(a.var(ddof=1)/len(a)+b.var(ddof=1)/len(b))
    df = (a.var(ddof=1)/len(a)+b.var(ddof=1)/len(b))**2 / ((a.var(ddof=1)/len(a))**2/(len(a)-1)+(b.var(ddof=1)/len(b))**2/(len(b)-1))
    t = diff/se
    p = 2*stats.t.sf(abs(t), df)
    q = stats.t.ppf(.975, df)
    return {"contrast": f"{label_a} - {label_b}", "difference": diff, "ci_low": diff-q*se, "ci_high": diff+q*se, "t": t, "df": df, "p": p}

tests = []
for dose in ["0.5", "1.0", "2.0"]:
    sub=tg[tg.dose==dose]
    tests.append(welch(sub[sub.supp=="OJ"].len, sub[sub.supp=="VC"].len, f"OJ@{dose}", f"VC@{dose}"))
for a,b in [("1.0","0.5"),("2.0","1.0"),("2.0","0.5")]:
    tests.append(welch(tg[tg.dose==a].len, tg[tg.dose==b].len, f"dose {a}", f"dose {b}"))
for t in tests: t["p_bonferroni"] = min(1.0, 6*t["p"])

summary = tg.groupby(["supp","dose"])["len"].agg(["count","mean","std"]).reset_index().to_dict("records")
results = {
    "simulation": {
        "seed": 20261007, "simulations": sims, "sample_size": n, "rate": rate,
        "sample_mean": float(means.mean()), "theoretical_mean": theory_mean,
        "sample_variance": float(means.var(ddof=1)), "theoretical_variance": theory_var,
        "sample_sd": float(means.std(ddof=1)), "theoretical_sd": float(np.sqrt(theory_var))
    },
    "toothgrowth_rows": int(len(tg)), "group_summary": summary, "tests": tests,
    "testing_note": "Welch two-sample t intervals/tests; six p-values also Bonferroni-adjusted."
}
(ROOT/"results.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
print(json.dumps(results, indent=2))
```